

K.SUJITH 2303A52346

BATCH 45

Suppose that you work for a company that receives hundreds of customer emails daily. Management wants to automatically classify emails into categories like "Billing", "Technical Support", "Feedback", and "Others" before assigning them to appropriate departments. Instead of training a new model, your task is to use prompt engineering techniques with an existing LLM to handle the classification.

Tasks to be completed are as below

a. Prepare Sample Data:

- Create or collect 10 short email samples, each belonging to one of the 4 categories.

b. Zero-shot Prompting:

- Design a prompt that asks the LLM to classify a single email without providing any examples.

- Example prompt:

“Classify the following email into one of the following categories: Billing, Technical Support, Feedback, Others. Email: ‘I have not received my invoice for last month.’”

c. One-shot Prompting:

- Add one labeled example before asking the model to classify a new email.

d. Few-shot Prompting:

- Use 3–5 labeled examples in your prompt before asking the model to classify a new email.

e. Evaluation:

- Run all three techniques on the same set of 5 test emails.
- Compare and document the accuracy and clarity of responses.

The screenshot shows a VS Code editor with a Python script in a file named `train_samples`. The script imports `OpenAI`, `re`, and `defaultdict` from `collections`. It defines an `OpenAI` client with an API key and a list of `train_samples` containing various customer feedback messages. A `test_samples` list is also defined with one message. The terminal window shows the command `python train_samples` being executed, resulting in a `ModuleNotFoundError: No module named 'openai'` error. The error message also shows the command `python` being executed in the terminal.

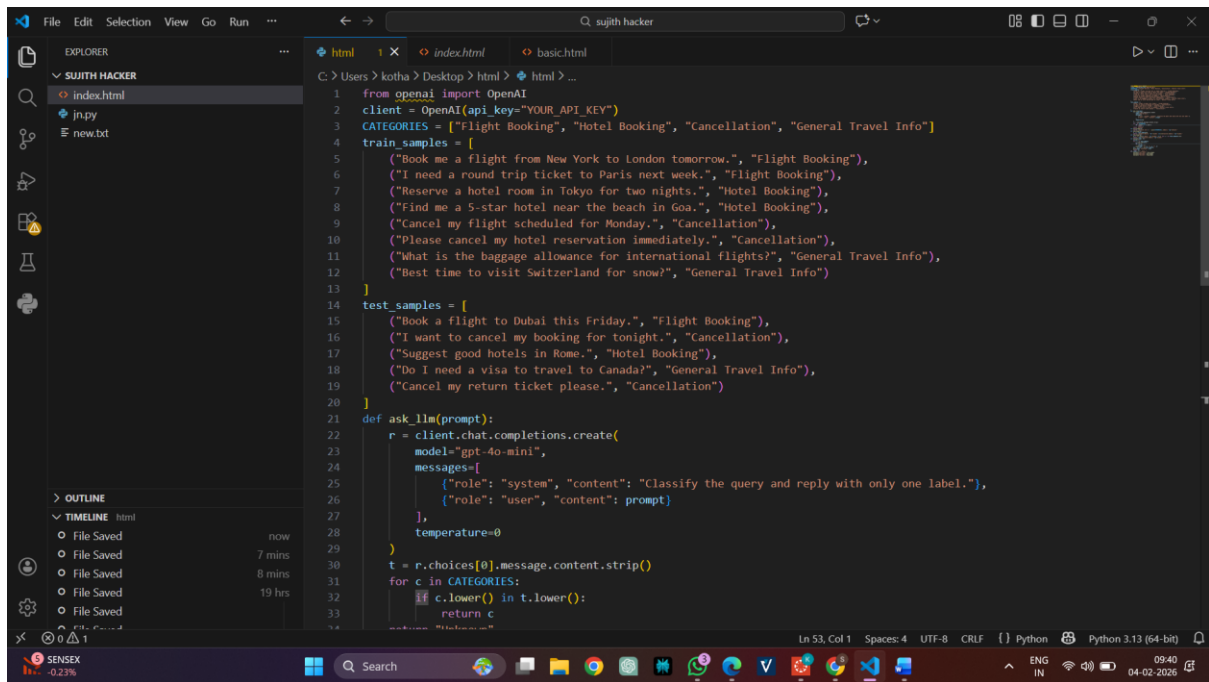
2. Travel Query Classification

Scenario:

A travel assistant must classify queries into Flight Booking, Hotel Booking, Cancellation, or General Travel Info.

Tasks:

- Prepare labeled travel queries.
- Apply Zero-shot prompting.
- Apply One-shot prompting.
- Apply Few-shot prompting.
- Compare response consistency.



```
1 from openai import OpenAI
2 client = OpenAI(api_key="YOUR_API_KEY")
3 CATEGORIES = ["Flight Booking", "Hotel Booking", "Cancellation", "General Travel Info"]
4
5 train_samples = [
6     ("Book me a flight from New York to London tomorrow.", "Flight Booking"),
7     ("I need a round trip ticket to Paris next week.", "Flight Booking"),
8     ("Reserve a hotel room in Tokyo for two nights.", "Hotel Booking"),
9     ("Find me a 5-star hotel near the beach in Goa.", "Hotel Booking"),
10    ("Cancel my flight scheduled for Monday.", "Cancellation"),
11    ("Please cancel my hotel reservation immediately.", "Cancellation"),
12    ("What is the baggage allowance for international flights?", "General Travel Info"),
13    ("Best time to visit Switzerland for snow?", "General Travel Info")
14]
15
16 test_samples = [
17     ("Book a flight to Dubai this Friday.", "Flight Booking"),
18     ("I want to cancel my booking for tonight.", "Cancellation"),
19     ("Suggest good hotels in Rome.", "Hotel Booking"),
20     ("Do I need a visa to travel to Canada?", "General Travel Info"),
21     ("Cancel my return ticket please.", "Cancellation")
22]
23
24 def ask_llm(prompt):
25     r = client.chat.completions.create(
26         model="gpt-4o-mini",
27         messages=[
28             {"role": "system", "content": "Classify the query and reply with only one label."},
29             {"role": "user", "content": prompt}
30         ],
31         temperature=0
32     )
33     t = r.choices[0].message.content.strip()
34     for c in CATEGORIES:
35         if c.lower() in t.lower():
36             return c
```

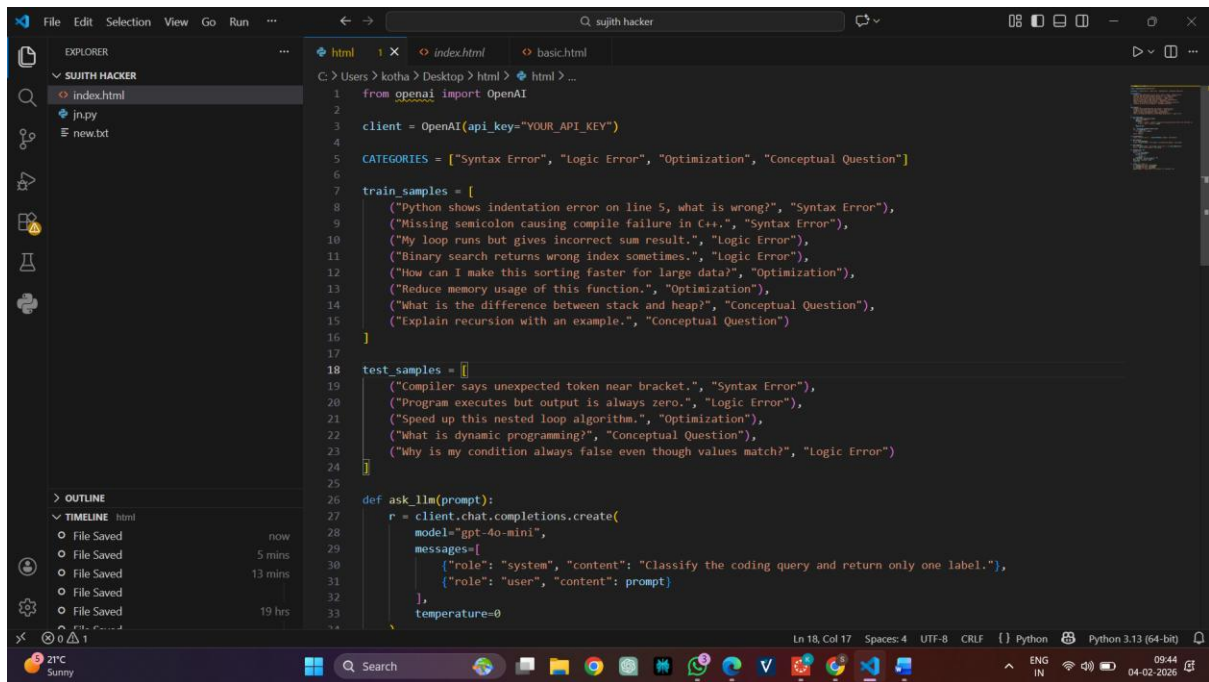
3. Programming Question Type Identification

Scenario:

A coding help chatbot must classify queries into Syntax Error, Logic Error, Optimization, or Conceptual Question.

Tasks:

- Prepare coding-related user queries.
- Perform Zero-shot classification.
- Perform One-shot classification.
- Perform Few-shot classification.
- Analyze improvements in technical accuracy.



```
1 from openai import OpenAI
2
3 client = OpenAI(api_key="YOUR_API_KEY")
4
5 CATEGORIES = ["Syntax Error", "Logic Error", "Optimization", "Conceptual Question"]
6
7 train_samples = [
8     ("Python shows indentation error on line 5, what is wrong?", "Syntax Error"),
9     ("Missing semicolon causing compile failure in C++.", "Syntax Error"),
10    ("My loop runs but gives incorrect sum result.", "Logic Error"),
11    ("Binary search returns wrong index sometimes.", "Logic Error"),
12    ("How can I make this sorting faster for large data?", "Optimization"),
13    ("Reduce memory usage of this function.", "Optimization"),
14    ("What is the difference between stack and heap?", "Conceptual Question"),
15    ("Explain recursion with an example.", "Conceptual Question")
16 ]
17
18 test_samples = [
19    ("Compiler says unexpected token near bracket.", "Syntax Error"),
20    ("Program executes but output is always zero.", "Logic Error"),
21    ("Speed up this nested loop algorithm.", "Optimization"),
22    ("What is dynamic programming?", "Conceptual Question"),
23    ("Why is my condition always false even though values match?", "Logic Error")
24 ]
25
26 def ask_llm(prompt):
27     r = client.chat.completions.create(
28         model="gpt-4o-mini",
29         messages=[
30             {"role": "system", "content": "Classify the coding query and return only one label."},
31             {"role": "user", "content": prompt}
32         ],
33         temperature=0
34     )
```

4. Social Media Post Categorization

Scenario:

A social media analytics tool must classify posts into Promotion, Complaint, Appreciation, or Inquiry.

Tasks:

1. Prepare sample social media posts.
2. Use Zero-shot prompting.
3. Use One-shot prompting.
4. Use Few-shot prompting.
5. Analyze informal language handling.

